

An Analysis of Optimistic and Conservative Concurrency Control in Multi-Threaded Key-Value Stores

Aaditya Manoj Borse

borsea@uci.edu

University of California, Irvine
Irvine, California, USA

Aaron Kirk

ajkirk@uci.edu

University of California, Irvine
Irvine, California, USA

Abstract

As modern databases transition toward high-concurrency environments, the choice of concurrency control (CC) protocol becomes critical to system performance. This paper explores the implementation and evaluation of two distinct CC protocols: Optimistic Concurrency Control (OCC) and Conservative Two-Phase Locking (C2PL). Both are built atop a RocksDB-based storage layer. By evaluating these protocols under varying degrees of thread parallelism and data contention, we demonstrate that while OCC excels in low-contention scenarios by avoiding locking overhead, C2PL provides more stable throughput in high-contention environments. We detail specific implementation optimizations, including lock-free key lookups, log pruning, and jittered exponential backoff for C2PL. We also analyze edge-case behaviors, such as C2PL lock-thrashing under extreme parallel contention.

Keywords

Transaction Processing, RocksDB, Optimistic Concurrency Control, Two-Phase Locking, Multi-threading, Livelock Prevention

ACM Reference Format: Aaditya Manoj Borse and Aaron Kirk. 2026. An Analysis of Optimistic and Conservative Concurrency Control in Multi-Threaded Key-Value Stores. Irvine, CA, USA.

1 Introduction

Concurrent transaction processing is the backbone of high-performance database management systems. The primary challenge lies in balancing the need for high parallelism with the necessity of maintaining strict serializability. In this project, we design and implement a multi-threaded transaction layer over RocksDB to investigate the trade-offs between two fundamental CC approaches: detection-based (OCC) and prevention-based (C2PL).

Our system supports workloads where multiple worker threads concurrently execute transactions performing randomized reads and writes. We evaluate how the performance of these protocols scales as we increase thread parallelism and adjust contention parameters. Specifically, we manipulate the hot-key probability parameter p and the total size of the hotset to observe how data locality impacts validation failures and lock queuing.

2 Background

The underlying storage layer utilizes RocksDB, an embedded persistent key-value store. While RocksDB natively handles atomic single-key operations, it does not guarantee multi-key atomicity

or serializable isolation across concurrent threads. Our transaction layer bridges this gap.

OCC operates on the premise that conflicts are rare. It allows transactions to proceed without locks and validates correctness strictly at commit time. Conversely, C2PL operates on a pessimistic assumption. It requires all locks to be acquired upfront, sacrificing potential concurrency for guaranteed execution once the transaction logic begins.

3 System Design

The architecture is divided into a Storage Layer utilizing RocksDB and a multi-threaded Transaction Layer responsible for all CC logic, conflict resolution, and transaction lifecycle management.

3.1 Conservative Two-Phase Locking (C2PL)

Our C2PL implementation relies on a centralized LockManager. To eliminate hash map synchronization overhead during workload execution, the lock manager uses a `pre_populate` phase to allocate all necessary mutexes upfront. This guarantees $O(1)$ lock-free lookups for individual keys across threads.

- **Lock Acquisition and Deadlock Prevention:** Before executing any database operations, a transaction requests its entire required keyset via a unified acquisition function. This function sorts the keys lexicographically and removes duplicates. By ensuring all worker threads request locks in a globally consistent order, circular wait conditions are eliminated, making deadlocks structurally impossible.
- **Livelock Prevention:** We utilize non-blocking `try_lock()` operations. If any lock in the sorted keyset is unavailable, the transaction immediately releases all currently acquired locks to yield resources. To prevent livelocks, we implemented an exponential backoff strategy. The backoff duration is calculated as $1.0 \times 2^{\min(\text{retries}, 15)} \mu\text{s}$, capped at a maximum of $100\mu\text{s}$, and multiplied by a uniform jitter of $[0.5, 1.5]$. A livelock threshold of 20 retries triggers a fallback to purely randomized maximum backoffs to break persistent symmetry between threads.
- **Lock Release:** Locks are held strictly for the duration of the transaction logic. Once the transaction completes its reads, performs computations, and applies writes to RocksDB, the system calls `unlock()` on the exact vector of acquired keys.

3.2 Optimistic Concurrency Control (OCC)

In our OCC implementation, transactions execute concurrently without blocking. This is achieved by isolating the read and write operations into local memory spaces until commit time.

- **Read and Write Sets:** Each transaction instance maintains a local read-set and a local write-set. During execution, reads are fetched directly from RocksDB and recorded in the read-set alongside the system's global commit version number. Writes are buffered entirely within the local write-set map.
- **Sequential Validation Protocol:** Validation is protected by a single global mutex. When a transaction attempts to commit, it acquires this lock. A transaction validates successfully if its local read-set does not intersect with the global write-sets of any transactions that committed after its recorded start version.
- **Memory Optimization (Log Pruning):** To prevent the global commit log from consuming unbounded memory, the system tracks the minimum active read version across all thread slots. A `prune_log` function routinely purges log entries older than this minimum version, as they cannot conflict with any currently executing transaction.
- **Commit and Abort Execution:** If validation passes, the global system commit counter is incremented, and the transaction's write-set is appended to the commit log. To minimize the duration of the critical section, the actual disk writes to RocksDB are applied after the validation lock is released. If validation fails, the transaction is aborted, its local buffers are cleared, and the transaction retries using a jittered exponential backoff bounded at a maximum of 50 ms.

4 Evaluation

We conducted experiments using two distinct workloads to evaluate relative performance. **Workload 1** serves as a baseline, consisting of a simple two-key account transfer. **Workload 2** acts as a stress test, containing complex transaction templates including an order-processing simulation with four reads and four writes, and a payment simulation with three reads and three writes.

4.1 Aborts and Retries

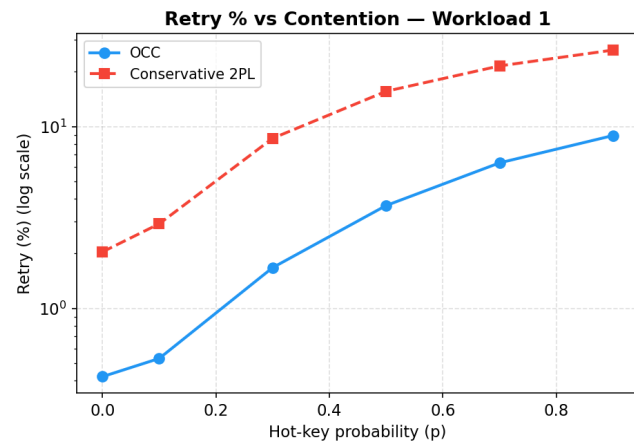


Figure 1: Retry Count vs. Contention (p) for Workload 1.

Figures 1 and 2 illustrate the raw retry counts as the hot-key probability scales. Under low contention, both protocols exhibited

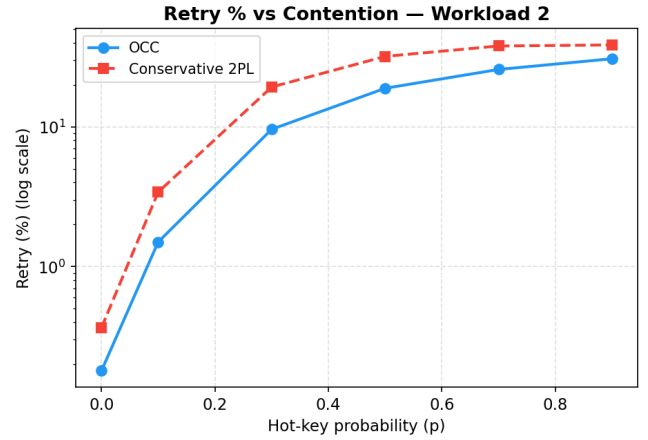


Figure 2: Retry Count vs. Contention (p) for Workload 2.

near-zero failure rates. However, as p approaches 1.0, OCC retry rates curve upward as overlapping write-sets invalidate active readers. Because Workload 2 contains larger read and write sets, its probability of intersection is inherently higher, causing more frequent OCC failures. C2PL retries scale exponentially on the graphs; however, a C2PL retry simply indicates the lock manager failed the upfront atomic acquisition phase and executed a microsecond-scale backoff. This avoids the heavy CPU penalty of OCC, which requires a full restart of the transaction logic.

4.2 Throughput Analysis

Throughput vs. Contention:

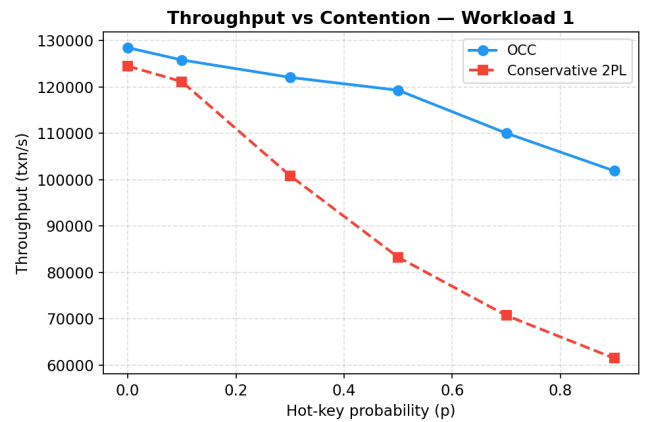


Figure 3: Throughput vs. Contention for Workload 1.

Under low contention ($p < 0.2$), OCC scales efficiently, fully leveraging its lock-free execution phase. Figures 3 and 4 demonstrate OCC outperforming C2PL initially. However, as p approaches 0.9, OCC performance decreases sharply. CPU cycles are exhausted executing logic that ultimately aborts during the validation phase. C2PL maintains more consistent throughput in this high-contention

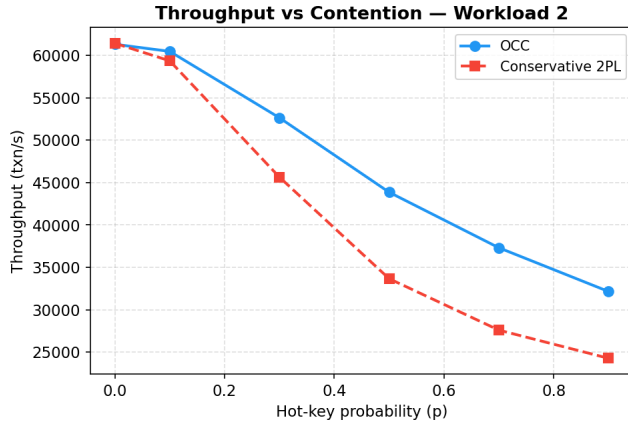


Figure 4: Throughput vs. Contention for Workload 2.

regime, eventually crossing over the OCC line to yield higher overall transactions per second.

Throughput vs. Hotset Size:

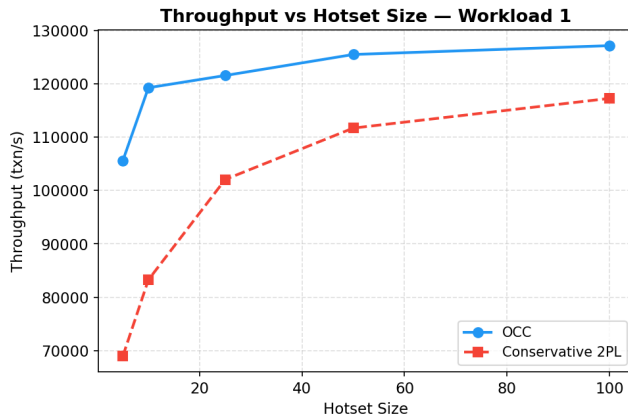


Figure 5: Throughput vs. Hotset Size for Workload 1.

We evaluated performance by fixing the contention probability and shrinking the hotset size to force data locality. As hotset size decreases on the x-axis, contention density increases. OCC's validation failure rate climbs accordingly, lowering throughput as seen in Figures 5 and 6. C2PL remains robust regardless of the hotset size because it guarantees execution safety upfront and relies on rapid backoffs rather than aborts.

Throughput vs. Threads (High Contention):

Under high contention (Figures 7 and 8), a system limitation becomes evident. OCC grew slowly; adding more parallel threads simply created more validation clashes, rendering the additional hardware resources ineffective. C2PL peaked early at 4 threads and then degraded as threads expanded to 16. When 16 threads fight to sequentially acquire exclusive locks within a highly restricted keyspace, they consistently hit the $100\mu s$ maximum backoff limit. This severe lock-thrashing confirms the vulnerability of C2PL to livelock-induced performance decay under extreme parallel stress.

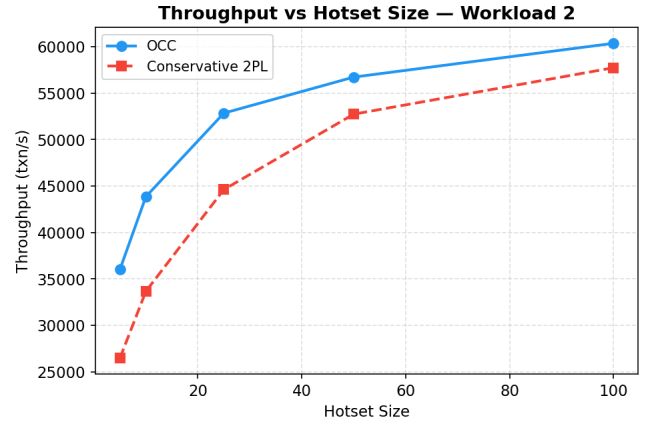


Figure 6: Throughput vs. Hotset Size for Workload 2.

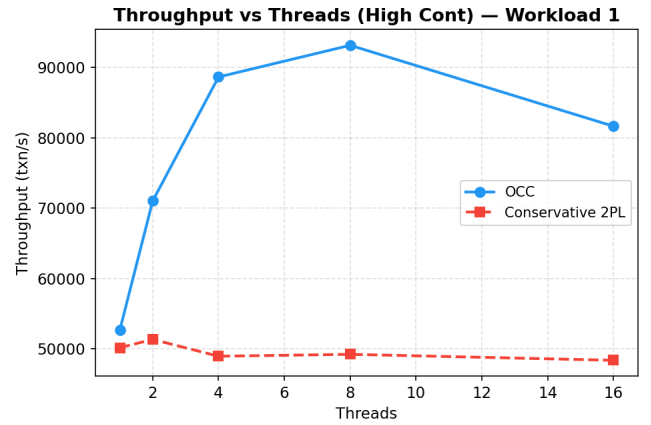


Figure 7: Throughput vs. Threads (High Contention, WL1).

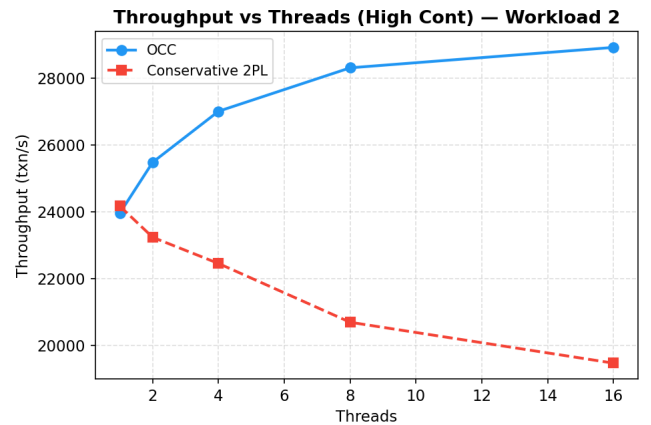


Figure 8: Throughput vs. Threads (High Contention, WL2).

4.3 Response Time

Response Time vs. Threads (Medium Contention):

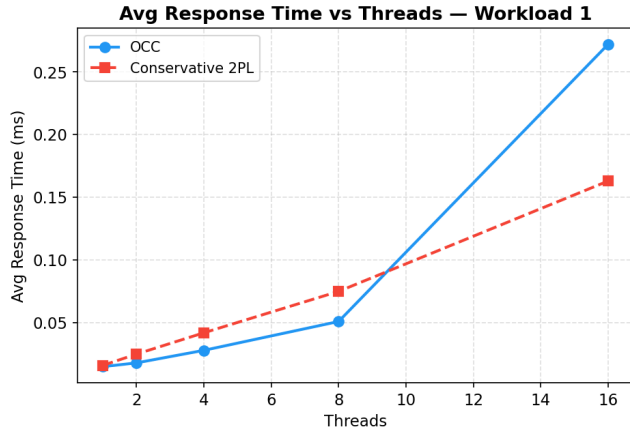


Figure 9: Response Time vs. Threads (Med Contention, WL1).

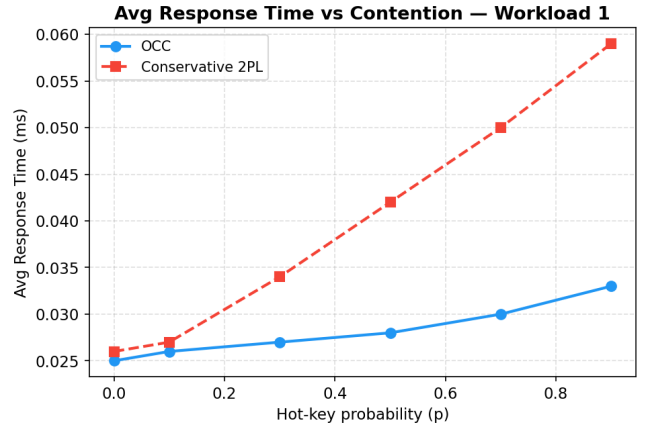


Figure 11: Response Time vs. Contention for Workload 1.

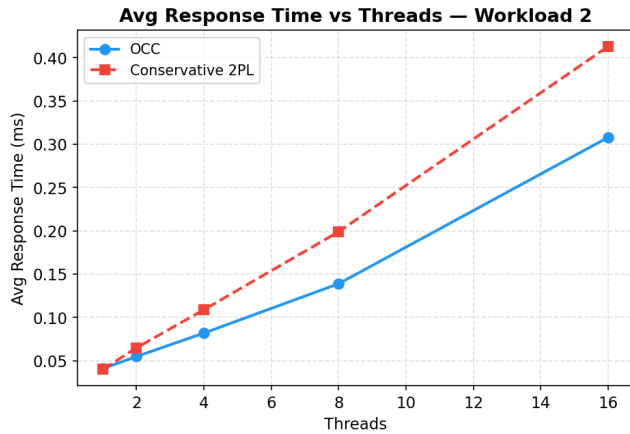


Figure 10: Response Time vs. Threads (Med Contention, WL2).

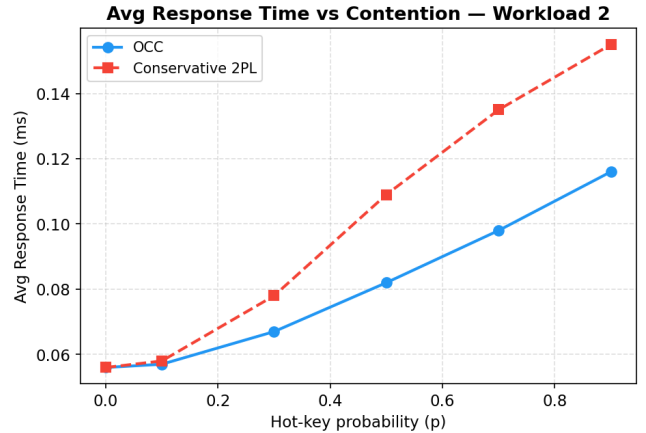


Figure 12: Response Time vs. Contention for Workload 2.

As thread counts increase (Figures 9 and 10), C2PL’s average response time grows because threads spend an increasing amount of time queued in the lock manager waiting for resources to free up. OCC maintains flatter baseline latency for successful transactions because it skips lock-acquisition overhead entirely, relying purely on rapid sequential validation.

Response Time vs. Contention:

However, as contention pushes towards 1.0 (Figures 11 and 12), OCC’s average response time begins to spike. The repeated aborts and backoffs accumulate for conflicted transactions, negatively impacting OCC’s average completion time for contended datasets.

Response Time Distribution:

The box plots (Figures 13 and 14) illustrate the latency tradeoffs in a structural context. Under OCC, the distributions are tightly packed at the bottom but feature significant upper whiskers representing p99 tail latency. These outliers are transactions starved by repeated validation failures, repeatedly hitting the 50 ms exponential backoff ceiling before successfully committing. C2PL exhibited slightly

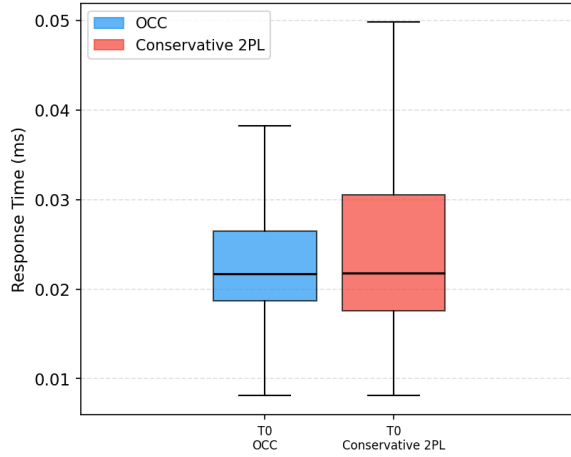
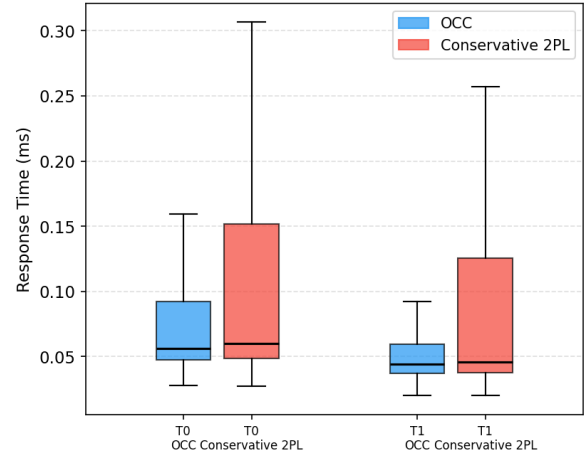
broader Interquartile Ranges (IQRs) and higher medians due to the constant $O(1)$ baseline locking overhead applied to every request. However, C2PL successfully suppressed the extreme tail latencies generated by full OCC logic restarts.

5 Conclusion

Our empirical results align with database concurrency theories. OCC is the optimal choice for latency-sensitive, read-heavy workloads with large keyspaces, where collisions are rare and locking overhead can be avoided. However, its sequential validation protocol limits vertical scaling under load. Conversely, Conservative 2PL is essential for constrained, hotspot-heavy datasets; by preventing transactions from executing without proper isolation, it limits worst-case performance decay. This advantage holds provided that thread parallelism does not exceed the threshold where lock-thrashing overwhelms the lock manager.

References

- [1] Facebook. RocksDB: A Persistent Key-Value Store for Flash and RAM Storage. <https://rocksdb.org/>.

Response Time Distribution per Template — Workload 1**Figure 13: Response Time Distribution for Workload 1.****Response Time Distribution per Template — Workload 2****Figure 14: Response Time Distribution for Workload 2.**

- [2] H. T. Kung and John T. Robinson. "On optimistic methods for concurrency control." ACM Transactions on Database Systems (TODS) 6.2 (1981): 213-226.
- [3] P. A. Bernstein and N. Goodman. "Concurrency Control in Distributed Database Systems." ACM Computing Surveys 13.2 (1981): 185-221.